Mississippi Valley State University

# CS 341 - Discrete Structures

## Practice Set 2: Approved Solutions

Negation, Conjunction, Disjunction, and Truth Tables

**Use these solutions responsibly**

Attempt each problem before reading its solution. Compare the reasoning and intermediate steps, not only the final answer. A response that differs in wording may still be correct if it has the same logical meaning and is fully justified. These solutions are for ungraded practice and may not be copied into graded work.

**Notation accepted in CS 341**

$\sim P$ and $\neg P$ both mean "not $P$." The operators $\wedge$, $\vee$, and $\oplus$ mean conjunction, inclusive disjunction, and exclusive disjunction. In code, the common equivalents are `not`/`!`, `and`/`&&`, and `or`/`||`.

## 1 Truth-Table Foundations

**1. Row counts**

| Variables | 1 | 2 | 3 | 5 |
|---|---|---|---|---|
| Rows required | **2** | **4** | **8** | **32** |

The general rule is $2^n$ **rows** for $n$ component statement variables.

**2. Standard order for two variables**

| $P$ | $Q$ |
|---|---|
| $T$ | $T$ |
| $T$ | $F$ |
| $F$ | $T$ |
| $F$ | $F$ |

**3. Standard order for three variables**

| $P$ | $Q$ | $R$ | $P$ | $Q$ | $R$ |
|---|---|---|---|---|---|
| $T$ | $T$ | $T$ | $F$ | $T$ | $T$ |
| $T$ | $T$ | $F$ | $F$ | $T$ | $F$ |
| $T$ | $F$ | $T$ | $F$ | $F$ | $T$ |
| $T$ | $F$ | $F$ | $F$ | $F$ | $F$ |

**4. Why four variables require $2^4$ rows**

**Approved solution**

Each variable has two possible truth values. By the multiplication principle, four variables have

$$2 \cdot 2 \cdot 2 \cdot 2 = 2^4 = \mathbf{16}$$

truth-value assignments. Equivalently, each row corresponds to one four-bit string if $T$ is represented by 1 and $F$ by 0.

**5. Defect in the proposed table**

**Approved solution**

The row $(T, T)$ **is duplicated**, and the row $(F, T)$ **is missing**. A complete table must contain each possible assignment exactly once. Because $(F, T)$ is absent, the proposed table cannot test the compound statement for that input case.

## 2 Negation

**6. Truth table**

| $P$ | $\sim P$ |
|---|---|
| $T$ | $F$ |
| $F$ | $T$ |

| No. | Original | Approved negation |
|---|---|---|
| 7 | The database is available. | **The database is not available.** |
| 8 | The file contains at least 20 records. | **The file contains fewer than 20 records.** For an integer count, this means at most 19. |
| 9 | $x \leq 7$ | $x > 7$ |
| 10 | The integer $n$ is even. | **The integer $n$ is odd.** Equivalently, $n$ is not even. |
| 11 | Exactly one process is running. | **It is not the case that exactly one process is running**; equivalently, zero processes or at least two processes are running. |

**12. Error analysis for the negation of $x < 5$**

**Approved solution**

(a) With $x = 5$, both $x < 5$ and the student's proposed $x > 5$ are false.

(b) The correct negation is $x \geq 5$.

(c) The student lost the equality case $x = 5$. A statement and its negation must have opposite truth values for every possible value.

**Quick scope check**

| Expression | Complete part affected by $\sim$ |
|---|---|
| $\sim P \wedge Q$ | $P$ **only** |
| $\sim (P \wedge Q)$ | **the entire expression** $P \wedge Q$ |
| $P \vee (\sim Q)$ | $Q$ **only** |

**Common error to repair**

Do not create a gap between a comparison and its negation. The negation of $<$ is $\geq$, the negation of $\leq$ is $>$, and similarly for the other inequalities.

# 3 Conjunction, Disjunction, and Exclusive Or

**13. Completed truth table**

| $P$ | $Q$ | $P \wedge Q$ | $P \vee Q$ | $P \oplus Q$ |
|---|---|---|---|---|
| $T$ | $T$ | $T$ | $T$ | $F$ |
| $T$ | $F$ | $F$ | $T$ | $T$ |
| $F$ | $T$ | $F$ | $T$ | $T$ |
| $F$ | $F$ | $F$ | $F$ | $F$ |

**14. Server state with $P = T$ and $Q = F$**

| Expression | Value | Contextual explanation |
|---|---|---|
| $P \wedge Q$ | $F$ | The server is online, but the backup is not current; both requirements do not hold. |
| $P \vee Q$ | $T$ | At least one component is true: the server is online. |
| $P \oplus Q$ | $T$ | Exactly one component is true. |

**15. Intended meaning**

| Item | Reason | Connective |
|---|---|---|
| (a) | Matching title, body, or both is permitted. | $\vee$ |
| (b) | Exactly one authentication method must be selected. | $\oplus$ |
| (c) | Either device or both devices may be used. | $\vee$ |
| (d) | A bit cannot be both 0 and 1. | $\oplus$ |

**16. Why the first rows differ**

**Approved solution**

Inclusive or, $P \vee Q$, means "at least one," so the both-true row is true. Exclusive or, $P \oplus Q$, means "exactly one," so the both-true row is false.

**17. Truth conditions**

(a) $P \wedge Q$ is true only when **both $P$ and $Q$ are true**.

(b) $P \vee Q$ is false only when **both $P$ and $Q$ are false**.

(c) $P \oplus Q$ is true only when **exactly one of $P$ and $Q$ is true**.

**18. Sample created rules**

**Approved solution**

**Inclusive or:** "Display a search result if the keyword appears in the title or the body; a result matching both is still displayed."
**Exclusive or:** "For this recovery step, select exactly one destination: the local drive or the network drive, but not both."
Other examples are acceptable if they explicitly handle the both-true case.

# 4 Translate Between English and Symbols

Let $P$ mean "The program compiles," $Q$ mean "The tests pass," and $R$ mean "The release is deployed."

| No. | Statement | Approved symbolic form |
|---|---|---|
| 19 | The program compiles and the tests pass. | $P \wedge Q$ |
| 20 | The program compiles, but the tests do not pass. | $P \wedge (\sim Q)$ |
| 21 | At least one is true: the tests pass or the release is deployed. | $Q \vee R$ |
| 22 | Exactly one is true: the program compiles or the tests pass. | $P \oplus Q$ |

**23. Translate** $(P \wedge Q) \vee R$

**Approved solution**

**"The program compiles and the tests pass, or the release is deployed."** The parentheses make $P \wedge Q$ one grouped component of the disjunction. A clearer wording is: "Either both the program compiles and the tests pass, or the release is deployed."

**24. Translate** $P \wedge \sim (Q \vee R)$

**Approved solution**

**"The program compiles, and it is not the case that the tests pass or the release is deployed."** This wording preserves the scope of the negation. Rewriting it with separate negations uses De Morgan's Laws, which are reserved for the next lecture.

**Additional translation drill**

(a) $\sim P \vee (Q \wedge R)$: **"The program does not compile, or both the tests pass and the release is deployed."**

(b) "The program does not compile, or both the tests pass and the release is deployed": $\sim P \vee (Q \wedge R)$.

(c) In $\sim P \vee Q$, only $P$ is negated. In $\sim (P \vee Q)$, the complete disjunction is negated.

**Common error to repair**

The word "but" normally has the same truth condition as "and"; it adds contrast, not a new logical connective. Thus "$P$, but not $Q$" is $P \wedge (\sim Q)$.

# 5 Evaluate and Build Truth Tables

**25. Assigned values $P = T$, $Q = F$, and $R = T$**

| Expression | Value | Expression | Value |
|---|---|---|---|
| $\sim P$ | $F$ | $P \wedge Q$ | $F$ |
| $\sim Q$ | $T$ | $P \vee Q$ | $T$ |
| $P \wedge (\sim Q)$ | $T$ | $(P \vee Q) \wedge R$ | $T$ |
| $\sim (P \vee Q)$ | $F$ | $P \oplus R$ | $F$ |

Two sample evaluation chains are

$$P \wedge (\sim Q) = T \wedge (\sim F) = T \wedge T = \boldsymbol{T},$$
$$(P \vee Q) \wedge R = (T \vee F) \wedge T = T \wedge T = \boldsymbol{T}.$$

**26. Truth table for $P \wedge (\sim Q)$**

| $P$ | $Q$ | $\sim Q$ | $P \wedge (\sim Q)$ |
|---|---|---|---|
| $T$ | $T$ | $F$ | $F$ |
| $T$ | $F$ | $T$ | $T$ |
| $F$ | $T$ | $F$ | $F$ |
| $F$ | $F$ | $T$ | $F$ |

It is true exactly when $P = T$ **and** $Q = F$.

**27. Truth table for $(\sim P) \vee Q$**

| $P$ | $Q$ | $\sim P$ | $(\sim P) \vee Q$ |
|---|---|---|---|
| $T$ | $T$ | $F$ | $T$ |
| $T$ | $F$ | $F$ | $F$ |
| $F$ | $T$ | $T$ | $T$ |
| $F$ | $F$ | $T$ | $T$ |

The only false input row is $P = T$, $Q = F$.

**Method check**

For Problem 26, the final column must be false whenever $P$ is false. For Problem 27, the final column must be true whenever $P$ is false because $\sim P$ is then true. These observations are useful checks, not replacements for the table.

# 6 Three-Variable Truth Table

**28. Truth table for $(P \vee Q) \wedge (\sim R)$**

| $P$ | $Q$ | $R$ | $P \vee Q$ | $\sim R$ | $(P \vee Q) \wedge (\sim R)$ |
|---|---|---|---|---|---|
| $T$ | $T$ | $T$ | $T$ | $F$ | $F$ |
| $T$ | $T$ | $F$ | $T$ | $T$ | $T$ |
| $T$ | $F$ | $T$ | $T$ | $F$ | $F$ |
| $T$ | $F$ | $F$ | $T$ | $T$ | $T$ |
| $F$ | $T$ | $T$ | $T$ | $F$ | $F$ |
| $F$ | $T$ | $F$ | $T$ | $T$ | $T$ |
| $F$ | $F$ | $T$ | $F$ | $F$ | $F$ |
| $F$ | $F$ | $F$ | $F$ | $T$ | $F$ |

**Approved solution**

(a) The expression is true in **3 rows**: $(T, T, F)$, $(T, F, F)$, and $(F, T, F)$.

(b) Whenever the expression is true, $R$ **must be false**, so $\sim R$ is true.

(c) At least one of $P$ and $Q$ must be true. They may also both be true.

(d) One computing interpretation is:

- $P$: the password is correct;
- $Q$: a security key is accepted;
- $R$: the account is locked.

Then access is granted when at least one authentication method succeeds and the account is not locked.

**Common error to repair**

The last row is false even though $\sim R = T$. A conjunction requires both components; $P \vee Q$ is false when $P = F$ and $Q = F$.

**Ready for Lecture 3**

You are ready to move on if you can rebuild Problems 13, 26, 27, and 28 without looking and explain why each final column has its pattern. Lecture 3 uses matching truth-table columns to define logical equivalence and develop De Morgan's Laws.

# 7 Logic in Code

**29. Find and repair the security bug**

Let $P$ mean `password_correct` and $L$ mean `account_locked`.

**Approved solution**

(a) The faulty condition is $P \vee (\sim L)$. The intended condition is $P \wedge (\sim L)$.

(b) Two input assignments exposing the bug are:

- $P = T$, $L = T$: faulty $T \vee F = T$, but intended $T \wedge F = F$;
- $P = F$, $L = F$: faulty $F \vee T = T$, but intended $F \wedge T = F$.

(c) The corrected Python condition is:

```
if password_correct and not account_locked:
    grant_access()
```

(d) For $P = T$, $L = F$, both the faulty and correct conditions are true. A test that checks only a case where the program behaves correctly cannot reveal how the operators differ in other rows.

**30. Sample designed condition**

**Approved solution**

Many answers are possible. One approved solution is:

- $P$: the password is correct;
- $B$: biometric authentication is accepted;
- $L$: the account is locked.

The mathematical condition is

$$\boldsymbol{(P \vee B) \wedge (\sim L)}.$$

The Python version is:

```
if (password_correct or biometric_accepted) and not account_locked:
    grant_access()
```

Two test assignments are:

- $(P, B, L) = (T, F, F)$ gives $(T \vee F) \wedge T = T$;
- $(P, B, L) = (F, T, T)$ gives $(F \vee T) \wedge F = F$.

A student-created answer is correct if the variables are defined, all three required connectives appear, the code matches the symbolic expression, and both evaluations are correct.

# Common Errors to Repair

| Common error | Better reasoning |
|---|---|
| Negating $x < 5$ as $x > 5$ | Include equality: $x \geq 5$. A statement and its negation must cover every case. |
| Treating $\vee$ as "exactly one" | Mathematical $\vee$ is inclusive. Use $\oplus$ for exactly one. |
| Ignoring parentheses | Identify the complete scope of each negation before evaluating. |
| Skipping intermediate columns | Compute one connective per column so an error can be located. |
| Testing only the normal code path | Test every Boolean input pattern required by the specification. |